B15 Embeddings and clustering

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B15 Embeddings and clustering 3
B15-L1 Sentence embeddings for open-ended answers 3
B15-L2 HDBSCAN, clusters with a noise label 7
B15-L3 Validate clusters before you name them 11
Track B best-practice checklist 15
Appendix: solutions and answer keys 16

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B15 Embeddings and clustering

Roadmap: Months 7 to 9: NLP with deep learning. SentenceTransformers, HDBSCAN and cluster validation.

B15-L1 Sentence embeddings for open-ended answers

Concept

Open-ended survey answers ("What could we improve?") arrive as thousands of short texts. A **sentence embedding model** turns each text into a fixed-length vector, so that texts with similar meaning get nearby vectors. With the sentence-transformers library:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
vectors = model.encode(answers, batch_size=64, normalize_embeddings=True)
```

`all-MiniLM-L6-v2` returns 384 numbers per text. With `normalize_embeddings=True` every vector has length 1, so the **cosine similarity** of two texts is their dot product: 1 for the same direction, 0 for unrelated.

Rules that keep embeddings usable:

- **One model per analysis.** Vectors from different models, or different versions of one model, live in different spaces. Never compare or mix them. Record the model name and version with the vectors.
- **Cache the vectors.** Encoding is the slow step. Store vectors with the text ID, the model name and a hash of the text, and re-encode only new or changed answers.
- **Clean lightly.** Strip whitespace and drop empty answers. Do not stem or remove stop words: the model was trained on natural text.
- **Check the language.** Many small models are English-only. Use a multilingual model for multi-country surveys, and test it on your own answers.
- **Handle zero vectors.** An empty string can produce a vector you should not compare. Filter before encoding, and guard any division by a vector's length.

Embeddings reflect the text the model was trained on, including its biases. Read samples of what the model calls "similar" before you trust any grouping built on it.

sentence-transformers needs PyTorch, which this app cannot run in the browser. This course's build did not run sentence-transformers, so the API details above come from the library's documentation and are marked as not run. The exercises use NumPy on vectors.

Why it matters

An analyst embedded wave 1 answers with one model and, a year later, wave 2 answers with a newer model under the same short name. Similarity between the waves came out low across the board, and the report said themes had "shifted dramatically". The texts had not changed much. The two sets of vectors came from different spaces.

Worked example: weak code

```
def most_similar(query, matrix):
    scores = matrix @ query
    return int(scores.argmax())
```

1. **Line 2.** A dot product equals cosine similarity only when all vectors have length 1.

Worked example: strong code

```
import numpy as np


def top_k_similar(query: np.ndarray, matrix: np.ndarray, k: int) -> list[int]:
    """Row indices of the k rows most cosine-similar to query, most similar first."""
    q_norm = np.linalg.norm(query)
    if q_norm == 0:
        raise ValueError("query vector has zero length")
    norms = np.linalg.norm(matrix, axis=1)
    sims = np.full(len(matrix), -np.inf)
    ok = norms > 0
    sims[ok] = (matrix[ok] @ query) / (norms[ok] * q_norm)
    return [int(i) for i in np.argsort(-sims, kind="stable")[:k]]
```

1. **Line 10.** Zero-length rows rank last instead of causing a division by zero.
2. **Line 12.** Cosine similarity that does not assume normalised input.

One model, recorded and cached. Compare with cosine similarity, and guard against zero vectors.

Recap card

- **Encode with sentence-transformers** SentenceTransformer(name).encode(texts, normalize_embeddings=True). Not run in this course's build.
- **Cosine similarity for unit vectors** The dot product. Otherwise divide by both lengths.
- **Mixing embedding models** Never. Each model defines its own space.
- **Cache key for a vector** Text ID, text hash, model name and version.

Practice

Exercise 1. Multiple choice (level 1)

You have vectors for 2025 answers from model A, and for 2026 answers from model B. Can you compare them?

- a) Yes, if both have 384 dimensions.
- b) No. Re-encode both years with the same model.
- c) Yes, after normalising both.
- d) Yes, by averaging them.

Why?

- r1) Each model defines its own space. The same position means different things in each.
- r2) Newer models are always better.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
import numpy as np

a = np.array([3.0, 4.0])
b = np.array([1.0, 0.0])
print(float(a @ b), round(float(a @ b / (np.linalg.norm(a) *
    np.linalg.norm(b))), 2))
```

Exercise 3. Fill the gap (level 2)

Normalise each row of a matrix to length 1.

```
norms = np.linalg.norm(matrix, axis=_____(1), keepdims=True)
unit = matrix / _____(2)
```

Exercise 4. Write the function (level 3)

Write `top_k_similar(query, matrix, k) -> list[int]`. Return the row indices of the `k` rows with the highest cosine similarity to `query`, highest first, as Python ints. Rows with zero length rank last. Raise `ValueError` if `query` has zero length. Do not assume the inputs are normalised.

Your code must pass these checks:

- Rank by cosine similarity, not raw dot product: row 0 is long but points the wrong way.
- Zero-length rows rank last, and indices are Python ints.
- A zero-length query must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Row indices of the k rows most cosine-similar to query.
def top_k_similar(
    query: np.ndarray,
    matrix: np.ndarray,
    k: int,
) -> list[int]: ...
```

Starter code:

```python
import numpy as np


def top_k_similar(query, matrix, k):
    return list(np.argsort(matrix @ query)[:k])
```

Exercise 5. Architecture design (level 4)

Plan embeddings for a quarterly tracker's open-ended question, in five languages.

Options: One multilingual model, tested on a sample from each language, An English-only model for all languages, A different model per language.

1. Model
2. What is stored with each vector
3. Next quarter

Sources

- SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2").encode(texts, batch_size=..., normalize_embeddings=True) returns 384-dimensional vectors. (unverified: sentence-transformers documentation and model card. Not run in this course's build.)

B15-L2 HDBSCAN, clusters with a noise label

Concept

k-means needs you to choose the number of clusters, and it puts every point in one. Open-ended answers rarely fit that: some themes are large, some small, and many answers belong to no theme at all. **HDBSCAN** finds dense groups of points, lets clusters differ in size and shape, and labels points in sparse regions as **noise**, `-1`.

scikit-learn has included it since version 1.3. With scikit-learn 1.8:

```
from sklearn.cluster import HDBSCAN

labels = HDBSCAN(min_cluster_size=15, copy=True).fit_predict(vectors)
```

- `min_cluster_size` (default 5) is the main setting: the smallest group you would call a theme. Raise it for fewer, larger themes.
- `min_samples` controls how conservative clustering is. Higher values label more points as noise.
- Pass `copy=True`. scikit-learn 1.8 warns that the default will change from `False` to `True` in 1.10, and `False` lets the estimator modify your input array.
- The default metric is Euclidean. For normalised embeddings (lesson B15-L1), Euclidean distance ranks pairs in the same order as cosine similarity.

Read the result before you use it:

- **Noise share.** On 600 points in four blobs plus 60 random points, this course's build labelled 12% as noise. For short survey answers a large noise share is normal, and it is information: those answers do not share a theme with enough others.
- **Number and size of clusters.** One giant cluster and many tiny ones usually means the settings or the embedding need work.
- **Samples.** Read 10 answers from each cluster before you name it.

High-dimensional embeddings are often reduced first, for example with UMAP or PCA, to make density easier to find. Record every setting and the random seed of any reduction step, because the clusters depend on them.

Why it matters

A team clustered 8,000 complaints with k-means and k = 12, because the dashboard had space for 12 tiles. Every complaint was forced into a theme. A cluster labelled "billing" contained 900 complaints about billing and 400

unrelated ones that were nearest to it. Its share of complaints was overstated by 40%. HDBSCAN on the same vectors marked most of the 400 as noise.

Worked example: weak code

```
labels = KMeans(n_clusters=12).fit_predict(vectors)
shares = pd.Series(labels).value_counts(normalize=True)
```

1. **Line 1.** k chosen by the dashboard, every answer forced into a cluster, no seed.
2. **Line 2.** Shares inflated by answers that belong nowhere.

Worked example: strong code

```
import numpy as np


def summarise_labels(labels: np.ndarray) -> dict[str, float | int | None]:
    """Cluster count, noise share and largest cluster share (of clustered points)."""
    n = len(labels)
    clustered = labels[labels != -1]
    if n == 0:
        return {"n_clusters": 0, "noise_share": None, "largest_share": None}
    sizes = np.unique(clustered, return_counts=True)[1]
    return {
        "n_clusters": int(len(sizes)),
        "noise_share": float((labels == -1).mean()),
        "largest_share": float(sizes.max() / len(clustered)) if len(clustered) > 0 else None,
    }
```

1. **Line 7.** Noise is reported separately, never counted as a cluster.
2. **Line 14.** Guarded: all noise means no clusters to compare.

Let density decide the clusters, keep noise as its own answer, and summarise counts, noise share and balance before naming themes.

Recap card

- **HDBSCAN noise label** -1: points in sparse regions. Report the share, never treat it as a cluster.
- **HDBSCAN main setting** min_cluster_size (default 5). min_samples makes it more conservative.
- **HDBSCAN in scikit-learn 1.8** sklearn.cluster.HDBSCAN. Pass copy=True: the default changes in 1.10.
- **Read a clustering** Noise share, number and size of clusters, then samples from each.

Practice

Exercise 1. Multiple choice (level 1)

HDBSCAN labels 30% of survey answers `-1`. What does that mean?

- a) The algorithm failed on those answers.
- b) Those answers are in sparse regions and share no dense theme with enough others.
- c) They form a 31st cluster.
- d) They are duplicates.

Why?

- r1) -1 is the noise label for points that no dense cluster claims.
- r2) HDBSCAN always drops 30%.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

Three NumPy tools count labels:

- `labels[labels != -1]` keeps the elements where the condition is true.
- `np.unique(a)` returns the distinct values, sorted.
- `np.bincount(a)` counts each non-negative integer: position `i` holds how often `i` appears.

```
a = np.array([0, 2, 2, 2])
np.unique(a)     # [0 2]
np.bincount(a)   # [1 0 3]: one 0, no 1s, three 2s
```

`(a == 2).mean()` is the share of elements equal to 2, because `True` counts as 1.

```
import numpy as np

labels = np.array([0, 0, 1, -1, 1, 1, -1, 0, 0, 0])
clustered = labels[labels != -1]
print(len(np.unique(clustered)), float((labels == -1).mean()),
    float(np.bincount(clustered).max() / len(clustered)))
```

Exercise 3. Fill the gap (level 2)

Cluster embeddings with HDBSCAN, smallest theme 20 answers, without modifying the input array.

```
labels = HDBSCAN(_____(1)=20, copy=_____(2)).fit_predict(vectors)
```

Exercise 4. Write the function (level 3)

Write `summarise_labels(labels) -> dict[str, float | int | None]` returning:

- `n_clusters`: distinct labels other than -1.
- `noise_share`: share of all points labelled -1, or `None` for an empty array.
- `largest_share`: size of the largest cluster over the number of clustered (non-noise) points, or `None` if there are none.

Your code must pass these checks:

- Exclude -1 from clusters. Noise share is over all points, largest share over clustered points.
- All noise: no clusters, noise share 1.0, largest share None.
- An empty array must not divide by zero.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Cluster count, noise share and largest cluster share.
def summarise_labels(
    labels: np.ndarray,
) -> dict[str, float | int | None]: ...
```

Starter code:

```python
import numpy as np


def summarise_labels(labels):
    return {"n_clusters": len(np.unique(labels))}
```

Exercise 5. Architecture design (level 4)

HDBSCAN on 5,000 complaint embeddings gives one cluster with 70% of clustered points, 40 tiny clusters, and 45% noise. Choose the next steps.

Options: Read samples from the big cluster, a few tiny ones, and the noise, Publish the 41 themes.

1. First step
2. Setting to try
3. Record

Sources

- sklearn.cluster.HDBSCAN defaults to min_cluster_size=5 and metric="euclidean"; scikit-learn 1.8 warns that copy will default to True in 1.10. (verified: Signature and warning observed on scikit-learn 1.8.0 and 1.9.1 in this repository.)

- Four blobs of 600 points plus 60 uniform points: HDBSCAN(min_cluster_size=15) found 4 clusters and 12% noise. (verified: Run with scikit-learn 1.8.0 in this repository (make_blobs random_state=0).)

B15-L3 Validate clusters before you name them

Concept

Any clustering algorithm returns clusters, even on random data. Before a cluster becomes a "theme" in a report, check three things.

1. Separation. The **silhouette score** compares each point's distance to its own cluster with its distance to the nearest other cluster. It runs from -1 to 1, higher is better. scikit-learn 1.8: `silhouette_score(X, labels)`. Compute it **without noise points**. The function treats `-1` as a cluster like any other. In this course's build, four blobs with HDBSCAN noise scored 0.70 with noise removed and 0.555 with noise counted as a cluster.

2. Stability. Rerun the whole pipeline on resampled data, or with different seeds, and compare the labels on the points both runs share. The **adjusted Rand index** (`adjusted_rand_score`) is 1 for identical groupings, around 0 for chance, and ignores label names: `[0, 0, 1, 1]` against `[1, 1, 0, 0]` scores 1.0. Themes that appear in only some runs are artefacts of the settings.

3. Meaning. Read 10 to 20 answers per cluster. Have two people name the clusters independently, then compare. A cluster nobody can name consistently is not a theme yet.

Report with each theme:

- its size and share, with noise reported separately,
- the silhouette and stability figures for the clustering as a whole,
- three verbatim example answers, chosen at random, not the most typical ones.

No single number proves clusters are real. Together, separation, stability and human reading make a defensible case. Keep the validation results in the run record with the settings (lesson B6-L2).

Why it matters

An insight team presented seven "customer mindsets" from a clustering of attitude statements. A year later a new team reran the same method on a fresh sample. Three of the seven mindsets reappeared. The other four had

been unstable from the start: a stability check, rerunning on bootstrap samples, would have shown them splitting and merging from run to run. Two product launches had been planned around mindsets that did not exist.

Worked example: weak code

```
score = silhouette_score(vectors, labels)
print("7 clear segments, silhouette", round(score, 2))
```

1. **Line 1.** Noise (-1) counted as a cluster, and one run taken as the truth.
2. **Line 2.** A single number presented as proof.

Worked example: strong code

```
import numpy as np
from sklearn.metrics import adjusted_rand_score, silhouette_score


def silhouette_without_noise(X: np.ndarray, labels: np.ndarray) -> float | None:
    """Silhouette over clustered points only. None if fewer than two clusters remain."""
    keep = labels != -1
    kept = labels[keep]
    n_clusters = len(np.unique(kept))
    if n_clusters < 2 or n_clusters >= len(kept):
        return None
    return float(silhouette_score(X[keep], kept))


def stability(labels_a: np.ndarray, labels_b: np.ndarray) -> float:
    """Agreement of two labelings of the same points, ignoring label names."""
    return float(adjusted_rand_score(labels_a, labels_b))
```

1. **Line 7.** Noise is not a cluster and does not enter the silhouette.
2. **Line 10.** silhouette_score needs between 2 and n - 1 clusters. Return None instead of raising.
3. **Line 17.** Compare runs on shared points. Label names do not matter.

Check separation without noise, stability across reruns, and meaning by reading, before any cluster gets a name.

Recap card

- **Three checks before naming clusters** Separation (silhouette), stability (ARI across reruns), meaning (read and name independently).
- **Silhouette and HDBSCAN noise** Exclude -1 first. silhouette_score treats it as a cluster (0.70 vs 0.555 in this lesson's example).

- **Adjusted Rand index** 1 for identical groupings, about 0 for chance, can be negative, ignores label names.
- **Examples on a theme slide** Random verbatims, not hand-picked ones.

Practice

Exercise 1. Multiple choice (level 1)

A clustering has silhouette 0.62. Is that enough to report its clusters as themes?

- a) Yes. Above 0.5 is proof.
- b) No. Also check stability across reruns and read samples from each cluster.
- c) No. Silhouette must be exactly 1.
- d) Yes, if the number of clusters matches the dashboard.

Why?

- r1) Separation in one run says nothing about whether clusters recur or mean anything.
- r2) Silhouette is never informative.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import adjusted_rand_score

print(adjusted_rand_score([0, 0, 1, 1], [1, 1, 0, 0]),
      adjusted_rand_score([0, 0, 1, 1], [0, 1, 0, 1]))
```

Exercise 3. Fill the gap (level 2)

Compute the silhouette over clustered points only.

```
keep = labels _____(1) -1
score = silhouette_score(X[keep], labels[_____(2)])
```

Exercise 4. Write the function (level 3)

Write `silhouette_without_noise(X, labels) -> float | None`. Drop points labelled -1. If fewer than 2 clusters remain, or as many clusters as remaining points, return `None`. Otherwise return `silhouette_score` on the remaining points as a float.

Your code must pass these checks:

- Compute the silhouette on non-noise points only.

- Fewer than two clusters must return None, not raise.
- As many clusters as points must return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Silhouette over clustered points only.
def silhouette_without_noise(
    X: np.ndarray,
    labels: np.ndarray,
) -> float | None: ...
```

Starter code:

```python
from sklearn.metrics import silhouette_score


def silhouette_without_noise(X, labels):
    return silhouette_score(X, labels)
```

Exercise 5. Architecture design (level 4)

You will present six themes from open-ended answers. Choose what goes on the methods slide.

Options: Silhouette without noise, with the noise share stated, Silhouette with noise counted as a cluster.

1. Separation
2. Stability
3. Examples per theme

Sources

- Silhouette 0.70 excluding noise and 0.555 including it for HDBSCAN on four blobs plus uniform noise; adjusted_rand_score([0,0,1,1],[1,1,0,0]) = 1.0 and ([0,0,1,1],[0,1,0,1]) = -0.5. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B15-L1 Sentence embeddings for open-ended answers

B15-L1 exercise 1

Hints. (1) What defines the meaning of a direction? (2) The model that produced it. (3) Use one model for everything you compare.

Answer. b) No. Re-encode both years with the same model. Because: r1) Each model defines its own space. The same position means different things in each.

Re-encode all texts with one model and record which one.

Common wrong answers.

- `a`: Matching dimensions do not make the spaces the same.
- `c`: Normalising fixes length, not the meaning of directions.
- `d`: Averaging vectors from different spaces produces meaningless numbers.
- `b|r2`: Right answer, wrong reason. The issue is consistency, not quality.

B15-L1 exercise 2

Hints. (1) Dot product: 3×1 + 4×0. (2) Lengths: 5 and 1. (3) 3 / 5.

Answer.

```
3.0 0.6
```

A dot product equals cosine similarity only for unit-length vectors. Normalise or divide by the lengths.

Common wrong answers.

- `0.6 0.6`: The raw dot product is 3.0. Only after dividing by the lengths (5 and 1) do you get 0.6.

B15-L1 exercise 3

Hints. (1) One length per row. (2) keepdims makes the shape broadcast. (3) `1` and `norms`.

Answer.

```
norms = np.linalg.norm(matrix, axis=1, keepdims=True)
```

```
unit = matrix / norms
```

`axis=1` gives each row's length, and `keepdims=True` keeps a column shape that divides row-wise. Guard rows with zero length.

Common wrong answers.

- g1 = `0`: axis=0 gives one norm per column. Each row needs its own length: axis=1.

B15-L1 exercise 4

Hints. (1) Divide dot products by both lengths. (2) Start similarities at -inf, fill only rows with non-zero length. (3) `np.argsort(-sims, kind="stable")[:k]`, then convert to int.

Answer.

```python
import numpy as np


def top_k_similar(query: np.ndarray, matrix: np.ndarray, k: int) -> list[int]:
    """Row indices of the k rows most cosine-similar to query.

    Args:
        query: Vector of length d.
        matrix: Array of shape (n, d).
        k: Number of indices to return.

    Returns:
        Indices, most similar first. Zero-length rows rank last.

    Raises:
        ValueError: If query has zero length.
    """
    q_norm = np.linalg.norm(query)
    if q_norm == 0:
        raise ValueError("query vector has zero length")
    norms = np.linalg.norm(matrix, axis=1)
    sims = np.full(len(matrix), -np.inf)
    ok = norms > 0
    sims[ok] = (matrix[ok] @ query) / (norms[ok] * q_norm)
    return [int(i) for i in np.argsort(-sims, kind="stable")[:k]]
```

Cosine similarity ignores length, so a long vector pointing elsewhere does not win. Zero rows are kept but ranked last.

B15-L1 exercise 5

Hints. (1) One space for all comparisons. (2) What makes a cached vector reusable? (3) Mixing models breaks comparisons.

Answer.

- One multilingual model puts all languages in one space. Test it on your data.
- Store the identity of text and model, so vectors can be reused and audited.
- Reuse vectors and keep the model fixed. Changing model means re-encoding everything.

Fix the model, record identities, and reuse vectors only within one model.

Common wrong answers.

- model = `per_lang`: Different models give different spaces. Answers in different languages could not be compared.
- model = `english`: An English-only model maps other languages poorly. Test before you trust it.
- update = `newest`: New and old vectors would come from different spaces.
- store = `vector_only`: Without the model name and text hash, nobody knows which vectors can be reused.

B15-L2 HDBSCAN, clusters with a noise label

B15-L2 exercise 1

Hints. (1) What does -1 mean in HDBSCAN? (2) Where do those points sit? (3) In sparse regions.

Answer. b) Those answers are in sparse regions and share no dense theme with enough others. Because: r1) -1 is the noise label for points that no dense cluster claims.

Noise answers are real answers without a shared theme. Report their share and read some of them.

Common wrong answers.

- `a`: Noise is a result, not a failure.
- `c`: -1 is not a cluster. Counting it as one mixes unrelated answers.
- `d`: Duplicates are close together, so they would usually form or join a cluster.
- `b|r2`: Right answer, wrong reason. The noise share depends on the data and the settings.

B15-L2 exercise 2

Hints. (1) Remove -1 before counting clusters. (2) Noise: 2 of 10. (3) Largest cluster: 5 of the 8 clustered points.

Answer.

```
2 0.2 0.625
```

Two clusters, 20% noise, and the largest cluster holds 62.5% of clustered answers.

Common wrong answers.

- `3 0.2 0.5`: -1 is noise, not a cluster. Two clusters, and the largest (5 of 8) is 0.625 of clustered points.

B15-L2 exercise 3

Hints. (1) The smallest group you would call a theme. (2) Copy the input instead of changing it. (3) `min_cluster_size` and `True`.

Answer.

```
labels = HDBSCAN(min_cluster_size=20, copy=True).fit_predict(vectors)
```

`min_cluster_size` sets the smallest theme. `copy=True` protects your array and silences the scikit-learn 1.8 warning.

Common wrong answers.

- g1 = `n_clusters`: HDBSCAN has no n_clusters. The main setting is min_cluster_size.

B15-L2 exercise 4

Hints. (1) Handle the empty array first. (2) `clustered = labels[labels != -1]`. (3) `np.unique(clustered, return_counts=True)` gives cluster sizes.

Answer.

```
import numpy as np


def summarise_labels(labels: np.ndarray) -> dict[str, float | int | None]:
    """Cluster count, noise share and largest cluster share.

    Args:
        labels: Cluster labels, -1 for noise.

    Returns:
        n_clusters, noise_share (of all points) and largest_share (of
    clustered points).
```

```
    """
    n = len(labels)
    if n == 0:
        return {"n_clusters": 0, "noise_share": None, "largest_share":
None}
    clustered = labels[labels != -1]
    sizes = np.unique(clustered, return_counts=True)[1]
    return {
        "n_clusters": int(len(sizes)),
        "noise_share": float((labels == -1).mean()),
        "largest_share": float(sizes.max() / len(clustered)) if
len(clustered) > 0 else None,
    }
```

Noise is reported, not clustered. Guarded denominators keep all-noise and empty results honest.

B15-L2 exercise 5

Hints. (1) What does an unbalanced result tell you? (2) Which setting controls the smallest theme? (3) Reproducibility (lesson B6-L2).

Answer.

- Read before tuning: the samples show whether the big cluster is one theme or a mix.
- Adjust min_cluster_size (and min_samples) on purpose, one change at a time.
- Record every setting, so the themes can be reproduced next quarter.

Diagnose with samples, tune deliberately, and record everything that shaped the clusters.

Common wrong answers.

- diagnose = `publish`: One dominant cluster and 40 tiny ones rarely reflect real themes. Look first.
- tune = `kmeans`: k-means forces all 45% noise into themes, inflating their shares.
- record = `final`: Without the settings, nobody can rebuild or compare the themes.

B15-L3 Validate clusters before you name them

B15-L3 exercise 1

Hints. (1) Would the same clusters appear on a new sample? (2) Can people name them? (3) Three checks, not one.

Answer. b) No. Also check stability across reruns and read samples from each cluster. Because: r1) Separation in one run says nothing about whether clusters recur or mean anything.

Separation, stability and meaning together. One number is a start, not a conclusion.

Common wrong answers.

- `a`: No threshold turns one number into proof. Check stability and meaning too.
- `c`: 1 is almost never reached on real data. Good separation is not the only test.
- `d`: A dashboard layout is not evidence about the data.
- `b|r2`: Right answer, wrong reason. Silhouette is useful. It is not sufficient.

B15-L3 exercise 2

Hints. (1) Does ARI care what the labels are called? (2) Are the groupings the same in the first case? (3) The second is worse than chance.

Answer.

```
1.0 -0.5
```

ARI compares groupings, not names. It can go below 0 when agreement is worse than chance.

Common wrong answers.

- `0.0 -0.5`: ARI ignores label names. Swapping 0 and 1 gives the same grouping: 1.0.
- `1.0 0.0`: The second grouping splits every pair the first one keeps together, which scores below chance: -0.5.

B15-L3 exercise 3

Hints. (1) Exclude the noise label. (2) Use the same mask for labels. (3) `!=` and `keep`.

Answer.

```
keep = labels != -1
score = silhouette_score(X[keep], labels[keep])
```

One mask for both arrays keeps points and labels aligned.

Common wrong answers.

- g1 = `==`: That keeps only the noise. Keep points whose label is not -1.

B15-L3 exercise 4

Hints. (1) Mask out -1 first. (2) Count clusters among the remaining labels. (3) silhouette_score needs 2 <= clusters <= n - 1.

Answer.

```python
import numpy as np
from sklearn.metrics import silhouette_score


def silhouette_without_noise(X: np.ndarray, labels: np.ndarray) -> float | None:
    """Silhouette over clustered points only.

    Args:
        X: Feature matrix.
        labels: Cluster labels, -1 for noise.

    Returns:
        The silhouette score, or None when it is undefined.
    """
    keep = labels != -1
    kept = labels[keep]
    n_clusters = len(np.unique(kept))
    if n_clusters < 2 or n_clusters >= len(kept):
        return None
    return float(silhouette_score(X[keep], kept))
```

Noise is not a cluster, so it stays out. Guarding the cluster count turns an exception into a clear "undefined".

B15-L3 exercise 5

Hints. (1) What does noise do to silhouette? (2) Would the themes come back on new data? (3) Which examples are honest?

Answer.

- Silhouette on clustered points, with noise reported beside it.
- Stability across reruns, naming the themes that do not recur.
- Random verbatims, so readers see what a theme typically contains.

Honest methods show separation, stability and representative evidence.

Common wrong answers.

- separation = `sil_all`: Noise is scattered by definition. Counting it as a cluster drags the score down and mixes two messages.
- stability = `skip`: A clustering can be well separated in one run and different in the next.

- examples = `best`: Hand-picked examples make every theme look cleaner than it is.